Requirement on academic honesty (COMP2090SEF):

1.Each group must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP2090SEF

Data Structures, Algorithms And Problem Solving

Course Project Report

Group 48

Submission date: 17 April 2026

We declare that:

(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged;

(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;

(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;

(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

Name	SID
Hung Ki Ip	14155790
Wong Sze Ching	13941636
Chan Yu Cho	14052578

2. The group must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others

3. All members are liable to the plagiarism incidence, even if a free-rider.

Introduction

This report documents the design and implementation of a restaurant ordering system built in Python(The system is implemented in Python with a GUI built using Tkinter (ttk widgets)). The project incorporates one new data structure - DataFrame and one new algorithm —Regular Expression Pattern Matching, both of which are central to the system's functionality. The following sections introduce these topics, explain their roles, and describe their implementations within the project.

Chosen Data Structure: DataFrame

A DataFrame is a two-dimensional, tabular data structure provided by the pandas library in Python. It organises data into rows and columns, where each column carries a unique label, making it conceptually similar to a spreadsheet or a relational database table. Unlike Python's built-in lists or dictionaries, a DataFrame allows each column to store a different data type simultaneously, such as integers, floating-point numbers, and strings within the same structure.

Implementation in the System.

In the restaurant ordering system, a DataFrame is used to load and manage the entire menu dataset from the external Excel file menu_data.xlsx. Upon application startup, the pandas function pd.read_excel() is called with the openpyxl engine to parse the spreadsheet and store its contents as a DataFrame object. Each row in the DataFrame represents a single menu item, and the columns store attributes such as the item's Traditional Chinese name, Simplified Chinese name, English name, price, category, and menu type (breakfast, lunch, afternoon tea, or dinner). This structure allows the system to access any attribute of any menu item directly by column label, which is far more readable and maintainable than using numerical indices on a nested list.

Chosen Algorithm：Regular Expression Pattern Matching

Regular Expression (Regex) pattern matching is a formal algorithmic technique used to determine whether a given string conforms to a specified pattern. Unlike simple string comparison methods such as == or in, regular expressions can describe complex structural constraints using a compact pattern syntax, and the matching process is carried out by an underlying engine based on Finite Automaton theory. Specifically, when a regular expression pattern is compiled, it is first translated into a

Non-deterministic Finite Automaton (NFA), which may then be converted into a Deterministic Finite Automaton (DFA). During the matching phase, the input string is processed character by character, driving the automaton through a series of state transitions. If the automaton reaches an accepting state upon consuming the entire input, the match is successful; otherwise, it fails.

Implementation in the System

In the restaurant ordering system, regular expression pattern matching is applied during the checkout process to validate the customer's phone number input. The system requires the phone number to be exactly eight consecutive digits, with no spaces, letters, or special characters, in accordance with the standard Hong Kong local phone number format. This validation is performed using Python's built-in re module, specifically the re.match() function, which attempts to match the given pattern against the beginning of the input string.

Reference:

1. Pandas for Data Handling

The pd.read_excel function and the conversion to lists are fundamental parts of the Pandas library.

- Official Documentation for read_excel: This is the best place to learn about parameters like sheet_name, header, and usecols.
    - Reference: Pandas read_excel Documentation
- Understanding DataFrames vs. Lists: Since your code uses .values.tolist(), you should study how Pandas converts tabular data into NumPy arrays and then into Python objects.
    - Reference: Pandas DataFrame.values
- File I/O Guide: This guide explains how to handle different file formats and pathing (like your W:\ drive path).
    - Reference: Pandas IO Tools Guide

2. Regular Expressions (Regex)

Your phone number validation relies on the re module. Regex has its own syntax that acts like a "language within a language."

- Python re Module Official Docs: This covers re.match(), re.search(), and the difference between them.
    - Reference: Python re Documentation
- Regex HOWTO: A more beginner-friendly guide provided by Python's creators specifically for pattern matching.
    - Reference: Python Regular Expression HOWTO
- Interactive Testing: To study how ^\d{8}$ works, use a sandbox where you can see the matching happen in real-time.
    - Tool: Regex101 (Set the flavor to Python).

Appendix:

DateFrame:

```python
import pandas as pd


def create_menu(self):
    global menu
    try:
        df = pd.read_excel("W:\comp gp project\menu_data.xlsx")
        menu = df.values.tolist()
        return True
    except Exception as e:
        print(f"Error loading menu: {e}")
        messagebox.showerror("Error", text.get("excel_load_error", "Failed to load menu"))
        return False
```

Component	Description
Library	pandas (aliased as pd)
Source Format	Microsoft Excel (.xlsx)
Internal Storage	Global menu list

Regular Expression Pattern Matching:

```python
import re
def checkout(self):
    global cart, orders, order_number, free_beverage_map
    global coupon_discount, current_coupon

    if not cart:
        messagebox.showinfo(text.get("cart_empty", "Empty"),
                            text.get("cart_empty", "Cart is empty"))
        return

    name = self.name_entry.get().strip()
    phone = self.phone_entry.get().strip()

    if not name:
        messagebox.showerror("Error", text.get("enter_name", "Enter your name"))
        return
    if not re.match( pattern: r'^\d{8}$', phone):
        messagebox.showerror("Error", text.get("invalid_phone", "8-digit phone required"))
        return
```

Symbol	Definition	Function in this App

^	Anchor Start	Ensures the match starts from the first character typed.
\d	Digit Metacharacter	Restricts input to numbers (0-9) only; no letters or symbols.
{8}	Quantifier	Forces the input to be exactly 8 characters long.
$	Anchor End	Ensures no extra numbers or spaces are added after the 8th digit.